SpendLog AI

Prompt Documentation

LLM Prompt Design for the Gemini Agentic Layer

Document No: DOC-03
Project: Agentic Expense Tracker (SpendLog AI)
Repository: https://github.com/Harish-S28/Agentic-expense-tracker
Prepared for: Project Documentation / README reference

1. Purpose & Scope

This document catalogs every prompt template sent to the Google Gemini API by **AIAgent** (ai_agent.py), the exact runtime context injected into each prompt, the expected output contract, and the rule-based fallback that activates when Gemini is unavailable. Each prompt is an f-string built at call time from live application state (user profile, expense data, budget figures) — there is no static prompt file; prompts are constructed dynamically inside four private methods, each paired with a public method and a deterministic twin.

2. Model Selection Strategy

Before any prompt is sent, **AIAgent.__init__** performs a one-time capability probe: it iterates over **['gemini-2.5-flash', 'gemini-2.0-flash', 'gemini-1.5-flash', 'gemini-flash-latest']** and issues a 1-token test call (**'Ping'**) against each. The first model to respond without error is bound to **self.model** and used for all four prompt types for the remainder of the process lifetime. If every model fails (invalid key, quota, network), **has_gemini** remains False and all subsequent calls route to rule-based methods.

3. Per-Expense Suggestion Prompt

Method: `_gemini_suggestion()` **Triggered by:** POST /api/ai/suggest, right after an expense is logged

Runtime inputs injected into the prompt:

Field	Source	Example value
name, profession	user_profile table	'Asha', 'Student'
category, amount, note	Submitted expense payload	'Food', 350, 'lunch with friends'
monthly_total	SUM(amount) this month	8400
category_total	SUM(amount) this category, this month	2200
monthly_budget	budget_settings for current month	6000 or 'Not set'

Prompt template (as constructed in code):

```
You are a friendly personal financial AI for {name}, who is a {profession}.

They just logged: {category} expense of Rs.{amount} [-- Note: {note}].{monthly_context}

Respond in 3-4 sentences MAX:
1. Briefly acknowledge the expense with a warm, specific observation.
2. Ask ONE relevant question about this specific expense.
3. Give ONE practical money-saving tip for this category, tailored to their profession.
4. If monthly spending in {category} looks high, gently suggest reducing -- but be
encouraging.

Tone: conversational, friendly, specific. Use Indian context (Rs., Indian examples).
Plain text only -- no bullet points, no markdown.
```

Output contract: A 3-4 sentence plain-text string, no markdown, Indian-rupee denominated.

Fallback if Gemini unavailable: _rule_suggestion() selects a random tip from **CATEGORY_TIPS[category]['high'|'normal']** based on whether category_total exceeds a threshold, plus a fixed follow-up question from the same category's 'question' key.

4. Daily Budget Tip Prompt

Method: `_gemini_budget_tip()` **Triggered by:** GET /api/budget/status, on every dashboard load once a budget is set

Runtime inputs injected into the prompt:

Field	Source	Example value
name, profession	user_profile table	'Ravi', 'Employee'
base_daily	monthly_budget / days_in_month	200
cumulative_carry	Sum of (base_daily - actual spend) for every prior day this month	+340 or -120
effective_limit	base_daily + cumulative_carry	540
today_spent, over	SUM(amount) today; today_spent > effective_limit	610, True

Prompt template (as constructed in code):

```
Financial AI for {name} ({profession}).

Budget today:
- Base daily limit: Rs.{base_daily}
- Carry-over: {carry_desc}
- Today's effective limit: Rs.{effective_limit}
- Spent so far today: Rs.{today_spent}
- Status: [OVER BUDGET / Within budget]

Write a 1-2 sentence personalized tip. Be specific, warm, actionable. Use Rs. No markdown.
```

Output contract: A 1-2 sentence plain-text coaching message referencing the exact rupee figures.

Fallback if Gemini unavailable: **_rule_budget_tip()** branches on over/cumulative_carry sign to return one of four templated sentences (over-budget, bonus carry-over, penalty carry-over, or neutral status).

5. Chatbot Prompt

Method: `_gemini_chat()` **Triggered by:** POST /api/ai/chat, on every user message in the AI chat drawer

Runtime inputs injected into the prompt:

Field	Source	Example value
name, profession	user_profile table	'Meera', 'Parent'
today_spent, monthly_total	Live SQL aggregates for today / this month	150, 5200
monthly_budget, effective_limit	budget_settings + carry-over calc	6000, 205
top_categories	Top-3 categories this month with totals	'Food (2200), Transport (900), Bills (700)'
chat_history	Last 10 turns from client, mapped to Gemini roles	[{'role':'user',...}, ...]

Prompt template (as constructed in code):

```
You are a personal financial AI assistant for {name}, a {profession}.

Financial snapshot:
- Today's spending: Rs.{today_spent}
- This month's total: Rs.{monthly_total}
- Monthly budget: {monthly_budget or 'Not set'}
- Today's effective daily limit: {effective_limit or 'Not set'}
- Top spending categories: {top_categories}

Answer helpfully, concisely (3-5 sentences), with Indian financial context.
Be encouraging but honest about overspending. Make suggestions specific and actionable.

[sent via chat.start_chat(history=...) then send_message(f'{sys_ctx}\n\nUser: {message}')]
```

Output contract: 3-5 sentence conversational answer, grounded in the live financial snapshot, with multi-turn memory via Gemini's native chat history object.

Fallback if Gemini unavailable: _rule_chat() performs keyword matching on the user's message (e.g. 'today', 'month', 'budget', 'category', 'tip', 'analysis', 'on track') and returns the matching templated response; unmatched messages get one of three generic help prompts.

6. Monthly / Yearly Analysis Prompt

Method: `_gemini_analysis()` **Triggered by:** GET /api/ai/analysis, from the Analysis page

Runtime inputs injected into the prompt:

Field	Source	Example value
name, profession	user_profile table	'Karthik', 'Business'
month/year, total	Aggregated totals for the requested period	'2026-06', 14300
by_category	Category totals + transaction counts	'Food: 4200 (18 tx)'...
prev_total, change_pct	Previous month total and % change (month view only)	12800, +11.7
monthly_budget / by_month	Budget row (month view) or 12-month series (year view)	6000 or [...]

Prompt template (as constructed in code):

```
Write a detailed monthly financial analysis report for {name} ({profession}).

Month: {month}
Total Spent: Rs.{total}
{budget_str}
Previous Month: Rs.{prev_total} (Change: {change_pct}%)

Category Breakdown:
{cats_str}

Write 200-250 words covering:
1. Overall assessment of spending for a {profession} -- high/low/reasonable?
2. Top 2 categories (specific reduction tips for each)
3. Categories where spending can safely increase (if any)
4. Month-over-month comparison insight
5. ONE specific, actionable recommendation for next month

Tone: friendly, encouraging, specific. Indian financial context. Clear paragraphs. No bullet
lists.
[A parallel yearly-report prompt template applies the same structure to by_month
best/worst-month
and annual category data when period == 'year'.]
```

Output contract: A 200-250 word narrative report in prose paragraphs (no markdown bullet lists), covering all five required points in order.

Fallback if Gemini unavailable: **_rule_analysis()** builds an equivalent structured report programmatically: budget status sentence, month-over-month arrow indicator, top spending day, per-category tips pulled from **CATEGORY_TIPS**, and a profession-specific closing recommendation from **PROFESSION_CONTEXT**.

7. Prompt Engineering Patterns Used

- **Role priming:** every prompt opens by casting Gemini as 'a personal financial AI' for a named user, anchoring tone and domain before any data is presented.

- **Structured, numbered instructions:** each prompt gives Gemini an explicit ordered checklist (e.g. acknowledge → question → tip) rather than an open-ended request, tightly bounding output shape.
- **Explicit length constraints:** '3-4 sentences MAX', '1-2 sentences', '200-250 words' — these keep responses UI-appropriate (toast, tip banner, or full report) without post-hoc truncation.
- **Locale conditioning:** every prompt requests 'Indian context (Rs., Indian examples)', ensuring culturally relevant advice (e.g. Ayushman Bharat, SIPs, UPI cashback) rather than generic Western tips.
- **Format constraints:** 'Plain text only, no markdown' for chat/toast surfaces vs. 'clear paragraphs, no bullet lists' for the analysis report — matched to how each surface renders text in the frontend.
- **Persona conditioning:** the profession string is interpolated directly into the instruction ('tailored to their profession as a {profession}'), letting one prompt template serve five distinct personas without branching logic in the prompt itself.
- **Live-data grounding:** every numeric claim the model is asked to reason about (totals, limits, carry-over) is pre-computed in SQL and handed to the model as fact, preventing arithmetic hallucination.